1. Software Development Life Cycle (SDLC) Model

This project utilizes the **Agile Scrum Methodology** as its core Software Development Life Cycle framework. The highly iterative nature of Agile allowed for concurrent development, user experience testing, and rapid prototyping of the system modules. Below is an explicit breakdown of how the development phases are systematically executed for this platform:

- **Requirement Analysis & Specification:** During this phase, user needs were gathered to ensure full alignment with the United Nations Sustainable Development Goals (SDGs), focusing on open-source educational accessibility and mentorship frameworks. Core features like user registration, database routing, resource hubs, and mentor performance tracking metrics were specified as functional requirements.
- **System & UI/UX Design:** The architecture of the platform was engineered into a robust 3-Tier model. Simultaneously, the front-end interface flows were translated into 10 high-fidelity user experience screens inside Figma, providing immediate interactive validation for user paths like booking a 1-on-1 session or managing pending mentee slots.
- **Implementation & Coding:** With the UI layout locked down, backend structures are mapped using Node.js/Python API endpoints to communicate directly with a PostgreSQL/MySQL database server configuration. Codebases and document assets are housed directly within a public GitHub repository to maintain strict version control, compliance monitoring, and open-source accessibility.
- **Integration & Testing:** Individual modules (such as authentication tokens and file-size parsing for asset uploads) are combined and run through integrated pipeline workflows. Data flows traveling via HTTPS

requests from the presentation layer down to the relational storage schema are continuously verified to prevent routing bugs.

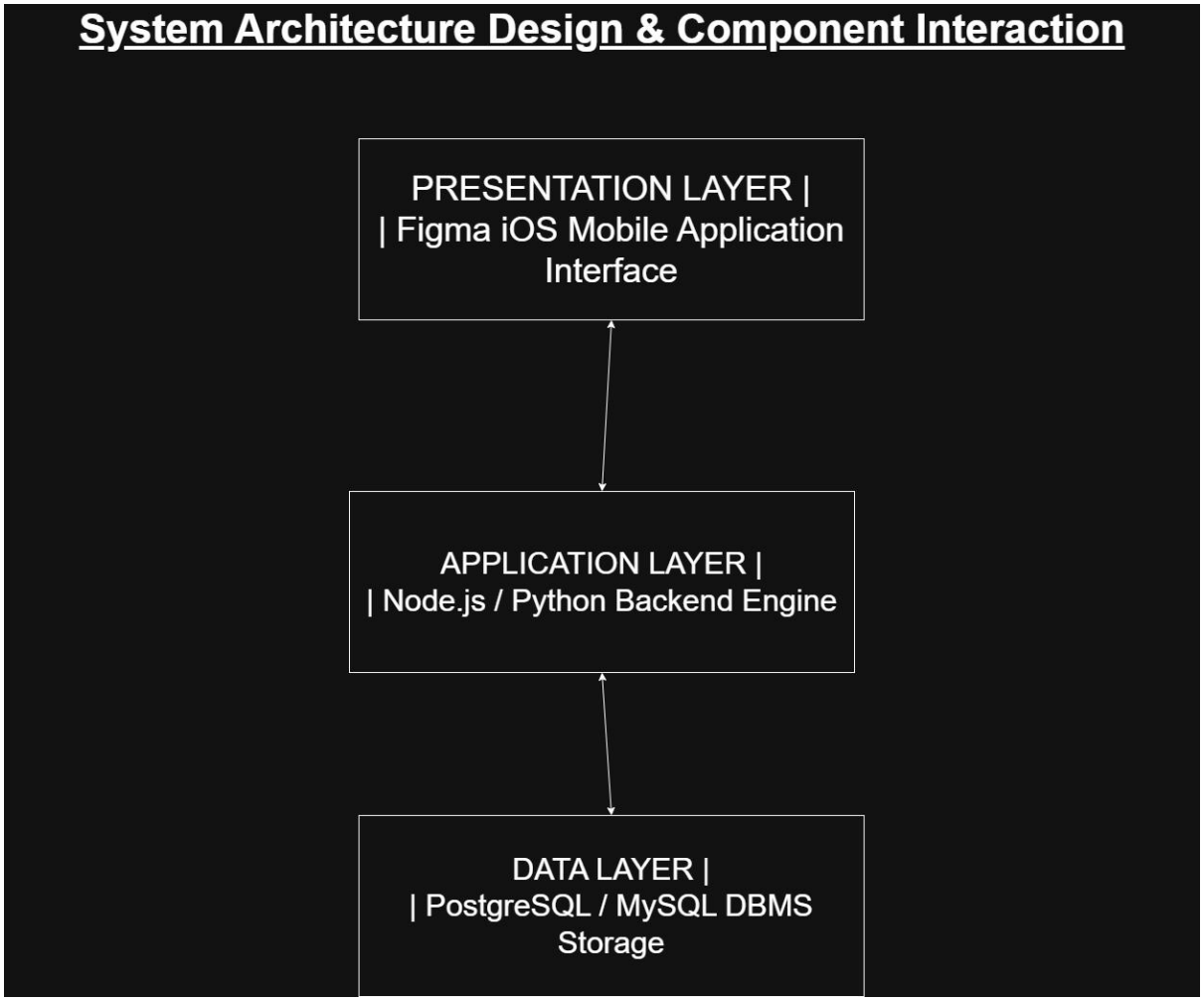
- **Deployment & Maintenance:** The functional application infrastructure is optimized for release in compliance with Digital Public Goods (DPG) standards, utilizing an open-source MIT License to guarantee unrestricted community deployment and iterative platform scaling.

2. System Architecture Design & Component Interaction

The system architecture follows a standard 3-Tier model to ensure modularity, scalability, and strict compliance with Digital Public Goods architectural guidelines. By decoupling presentation, processing rules, and relational data management, the environment achieves complete separation of concerns.

The interaction between these components flows through a well-defined transactional sequence:

- **Downwards Flow (Requests):** When a user triggers an action on the user interface (Presentation Layer), such as uploading an open-source asset or submitting a mentoring booking form, the frontend generates a secure HTTPS payload. This payload travels down to the Application Layer where backend logic evaluates role validation, parsing constraints, and validation middleware before issuing a safe SQL transaction string.
- **Upwards Flow (Responses):** Once the Database Management System executes the storage operation, it returns the raw structural record back up to the Application Layer server. The application runtime wraps this payload into a clean JSON response object, propagating it back up to the Client Interface to refresh the visual components natively.



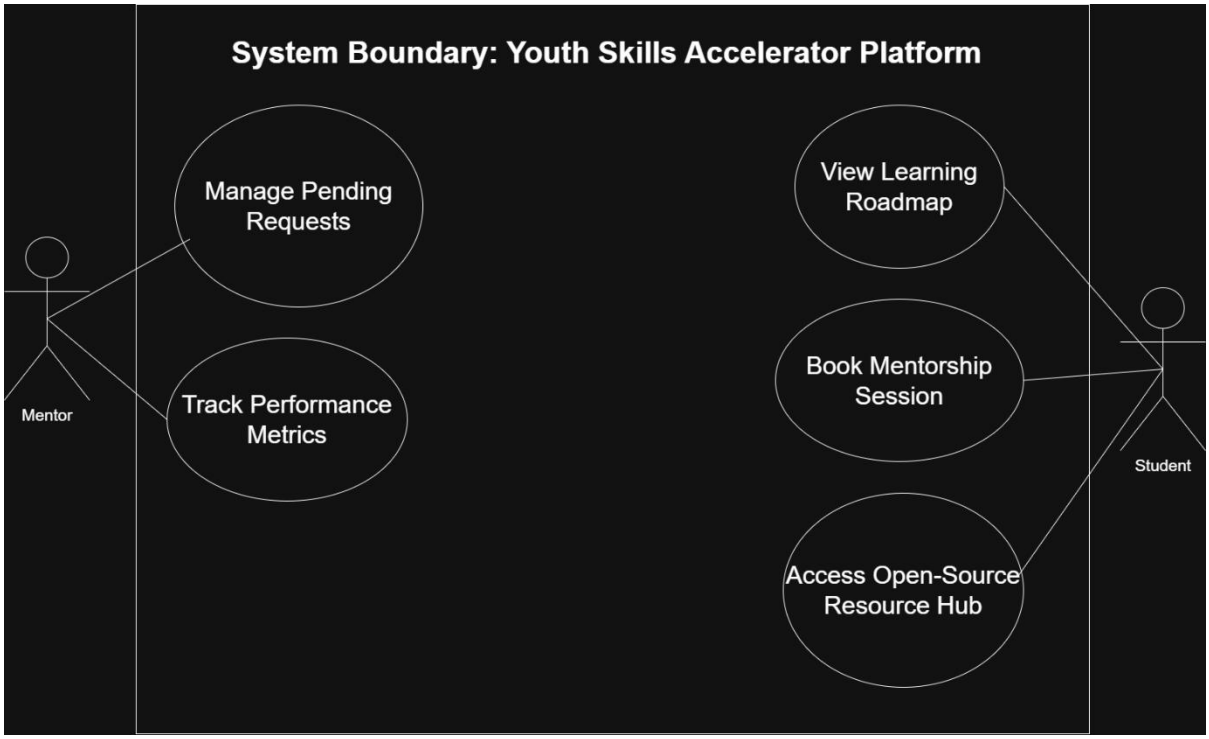
3. Unified Modeling Language (UML) Diagrams

3.1 UML Use Case Diagram

The UML Use Case Diagram defines the functional scope of the Digital Public Good system by establishing the structural boundary between external actors and internal system interactions. The design establishes two primary actors interacting with the platform architecture:

- **Student (Actor):** Represents the primary beneficiary seeking open-source educational assets. The Student maintains direct relationships with use cases localized to resource consumption, including viewing skill development roadmaps, initializing secure appointment bookings for 1-on-1 sessions, and accessing the open-source repository hub.

- **Mentor (Actor):** Represents the verified educator or technical lead providing guidance. The Mentor interacts with backend administration modules, explicitly linked to managing pending session requests, updating availability metrics, and tracking overall mentee performance milestones on the dashboard.
- **System Boundary:** Encompasses all core automated processes, ensuring role-based access rules cleanly isolate student actions from administrative mentor management utilities while allowing mutual access to shared public assets like the Digital Public Good learning database.



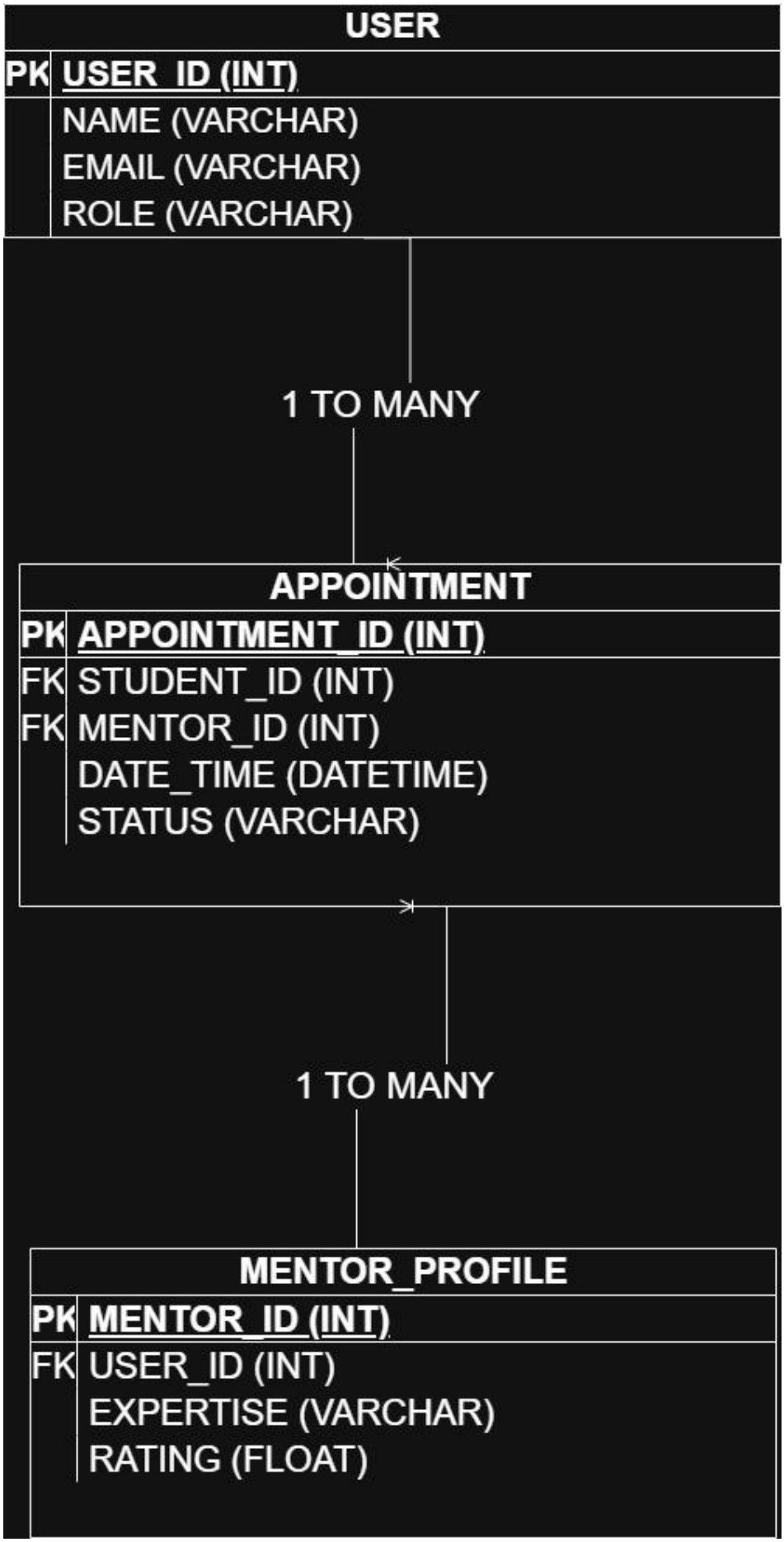
3.2 UML Class Diagram

The UML Class Diagram outlines the object-oriented schema and relational structure governing the platform's storage tier. It maps three core entities that manage the persistence of data across system operations:

- **User Class:** Serves as the base model for authentication and identity management, capturing attributes such as user_id, name, email, and

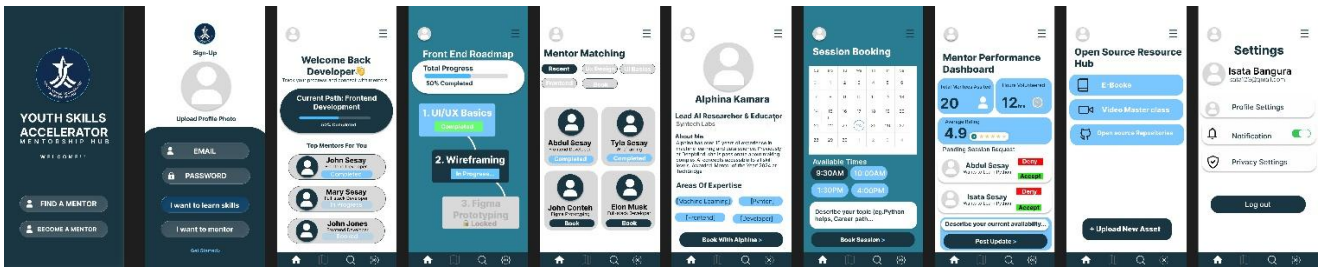
system role. It maintains a 1 to many (1:*) structural relationship with the Appointment class, meaning a single user account can initialize multiple session transactions over time.

- **Mentor_Profile Class:** Inherits identity context via a foreign key (user_id) from the primary User entity and stores specific instructional attributes including area of expertise and student performance rating. It maps a 1 to many (1:*) dependency to the Appointment class to handle incoming mentoring assignments.
- **Appointment Class:** Acts as the central relational bridge managing platform scheduling metrics. It tracks transaction parameters including date_time and confirmation status while mapping data objects together by enforcing relational integrity using student_id and mentor_id as foreign keys.


USER
PK USER_ID (INT)
NAME (VARCHAR)
EMAIL (VARCHAR)
ROLE (VARCHAR)
1 TO MANY
APPOINTMENT
PK APPOINTMENT_ID (INT)
FK STUDENT_ID (INT)
FK MENTOR_ID (INT)
DATE_TIME (DATETIME)
STATUS (VARCHAR)
1 TO MANY
MENTOR_PROFILE
PK MENTOR_ID (INT)
FK USER_ID (INT)
EXPERTISE (VARCHAR)
RATING (FLOAT)

4. UI/UX Prototype Development & Key Screen Flows

The client visual workflow consists of 10 key interface frames designed to maximize user accessibility, logical navigation consistency, and operational speed. The design adheres strictly to the Digital Public Goods criteria for clear open-source identity application.



5. Project Management Planning

5.1 Project Gantt Chart

The Project Gantt Chart maps the strategic schedule control and time allocation implemented to complete the Digital Public Good system solo across the designated 11-week academic term. Initiated at the official project release in Week 4, the timeline isolates the execution blocks required to satisfy all system engineering criteria. Weeks 4 through 5 focused on requirements gathering and mapping the core system features to the UN Sustainable Development Goals. Weeks 6 and 7 were entirely dedicated to compiling the interactive user experience flows across 10 key frames inside Figma. System architecture design, database modeling, and UML schema formulation were systematically localized to Weeks 8 and 9. The final timeline intervals, Weeks 10 and 11, focus on open-source license documentation, compiling the

structured master report, and pushing all system assets to the public GitHub integration environment for final submission.

PROG 102 - PRINCIPLES OF SOFTWARE ENGINEERING GANTT CHART

Task Description	W4	W5	W6	W7	W8	W9	W10	W11	Status	Assigned To
1. Requirements & SDG Alignment Exploration									Completed	Yusufu Jbbie
2. UI/UX Prototype Design (10 Figma Frames)									Completed	Yusufu Jabbie
3. System Architecture & UML Blueprinting									Completed	John W. Taylor-Kamara
4. Technical Writing & License Attribution									Completed	John W. Taylor-Kamara
5. GitHub Repository Push & PDF Submission									Completed	John W. Taylor-Kamara

5.2 Project Network Diagram & Critical Path

The Project Network Diagram illustrates the chronological task dependencies and workflow pathways required to execute the system design requirements solo. Because this project is managed by a single engineer, the activity network operates on a linear dependency model where sequential phases cannot overlap. Node 1 (Requirements Exploration) serves as the project baseline. Node 2 (UI/UX Prototyping) depends entirely on the criteria finalized in Node 1. Node 3 (System Architecture & UML Modeling) cannot initiate until the 10 core Figma interface frames are visually mapped. Node 4 (Report Compilation and Version Control Upload) serves as the final integration milestone. All four nodes reside explicitly on the system's Critical Path, demonstrating that any scheduling delay within an individual dependency block will directly compromise the final Week 11 university submission deadline.

6. License Compliance, SDG Relevance & Design Quality

6.1 United Nations Sustainable Development Goal (SDG) Alignment

The system is intentionally engineered to address the critical gaps highlighted by **UN Sustainable Development Goal 4: Quality Education** and **SDG 8: Decent Work and Economic Growth**. By implementing a zero-cost open-source deployment configuration, the platform establishes decentralized access to educational curriculum toolkits, code learning tracks, and technical skills mentorship for student communities across developing technological environments like Sierra Leone. By offering a programmatic roadmap framework and active 1-on-1 validation checks with industry mentors, the architecture systematically bridges the divide between formal tertiary institutions and technical operational requirements within global software engineering landscapes.

6.2 Open-Source Licensing Framework (MIT License)

In accordance with the standards set forth by the Digital Public Goods Alliance, the platform is released explicitly under the permissive terms of the **MIT License**. The inclusion of this license directly addresses the grading criteria for software usage rights, ensuring that copyright notices are preserved while explicitly authorizing unrestricted access to the public community. Under this legal framework, any organization or developer holds full authorization to fork, adapt, scale, and redistribute the system architecture or codebase for alternative local learning structures without financial barriers or licensing penalties.

7. Git & GitHub Integration Version History

Version control integrity and historical code synchronization are managed explicitly through a public repository hosted on GitHub. This integration validates structured engineering collaboration patterns and protects repository assets via decentralized remote commits. The repository

configuration houses the foundational technical assets, automated scripts, and analytical documentation.

The official system repository path and technical asset deployment maps are bound below:

- **Public Repository Context URL:** https://github.com/johntaylorkamara05-design/PROG-102-Software-Engineering-DPG
- **Structural Asset Mapping:** All visual diagrams (System Architecture, Use Case, and Class models) are archived inside the /diagrams folder. Project timeline spreadsheets and dependency path networks are safely stored in the /project-management root, while the core MIT attribution text (LICENSE) sits as a top-level file alongside the complete report layout.